

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Materia: Programación 1
Estudiante: Morales Aliaga Fabián
Fecha de entrega: 15 de junio de 2025

Índice

1. Marco Teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	3
1.4 Condicionales	4
1.5 Ordenamientos	4
1.6 Estadísticas básicas	4
1.7 Archivos CSV	4
2. Decisiones Técnicas y Arquitectura	5
2.1 Estructura del proyecto	5
2.2 Diagrama de flujo	5
2.3 Capturas de ejecución	6
3. Dificultades y Conclusiones	7
4. Bibliografía / Webgrafía	7

1. Marco Teórico

1.1 Listas

En Python, una **lista** es una estructura de datos ordenada y mutable que permite almacenar múltiples valores de cualquier tipo bajo un mismo nombre de variable. Se definen con corchetes y soportan operaciones como agregar, eliminar, recorrer y ordenar elementos.

En este proyecto, la lista principal *países* almacena todos los registros del dataset. Cada vez que se agrega, elimina o actualiza un país, se opera directamente sobre esta lista.

Ejemplo de uso en el proyecto:

```
países = [] # lista vacía inicial
países.append({'nombre': 'Argentina', 'poblacion': 45376763, ...})
```

1.2 Diccionarios

Un **diccionario** es una estructura de datos que almacena pares clave-valor. A diferencia de las listas, el acceso a los datos se realiza mediante claves descriptivas, lo que hace el código más legible y explícito.

Cada país del dataset se representa como un diccionario con cuatro claves: *nombre*, *poblacion*, *superficie* y *continente*. Esto permite acceder fácilmente a cualquier atributo, por ejemplo: *pais['poblacion']*.

1.3 Funciones

Las **funciones** son bloques de código reutilizables que encapsulan una tarea específica. En Python se definen con la palabra clave *def*. El principio de *una función = una responsabilidad* (cohesión) facilita el mantenimiento, la lectura y el testeo del código.

El proyecto aplica este principio de forma estricta: cada funcionalidad del menú corresponde a una función independiente, como *agregar_pais()*, *buscar_pais()*, *filtrar_por_continente()*, etc.

1.4 Condicionales

Las **estructuras condicionales** (*if / elif / else*) permiten que el programa tome decisiones en función del valor de una expresión lógica. Son fundamentales para validar entradas del usuario, verificar si un país ya existe antes de agregarlo, o controlar el flujo del menú principal.

Ejemplo: antes de agregar un país se verifica que no exista uno con el mismo nombre usando un condicional que recorre la lista con una expresión generadora.

1.5 Ordenamientos

Python provee la función nativa **sorted()**, que devuelve una nueva lista ordenada sin modificar la original. Acepta el parámetro *key* para indicar el criterio de ordenamiento y *reverse* para elegir entre ascendente y descendente.

En el proyecto se usa `sorted(paises, key=lambda p: p[campo], reverse=desc)` para ordenar por nombre, población o superficie, según elija el usuario.

1.6 Estadísticas básicas

Las estadísticas se calculan usando funciones nativas de Python: `max()`, `min()` y `sum()`, combinadas con expresiones generadoras. Esto evita la necesidad de librerías externas como NumPy o pandas, cumpliendo con los requerimientos de la materia.

Se calculan: país con mayor y menor población, promedio de población, mayor y menor superficie, promedio de superficie y cantidad de países por continente.

1.7 Archivos CSV

Un archivo **CSV** (*Comma-Separated Values*) es un formato de texto plano donde cada fila representa un registro y los campos se separan por comas. Python incluye el módulo estándar `csv` que facilita la lectura y escritura de estos archivos sin dependencias externas.

El programa usa `csv.DictReader` para cargar el archivo como una lista de diccionarios (cada fila queda mapeada a sus columnas automáticamente) y `csv.DictWriter` para guardar los cambios, manteniendo siempre la persistencia de datos.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura del proyecto

El proyecto se organiza en tres archivos en la raíz del repositorio:

- **gestion_paises.py** — Código fuente principal con toda la lógica del sistema.
- **paises.csv** — Dataset base con 40 países de todos los continentes.
- **README.md** — Documentación del proyecto con instrucciones de uso.

El archivo Python está modularizado en secciones claramente delimitadas mediante comentarios: carga/guardado, visualización, agregar, actualizar, buscar, filtros, ordenamiento, estadísticas, utilidades y menú principal.

2.2 Diagrama de flujo principal

El flujo de ejecución del programa sigue la siguiente secuencia:



2.3 Decisiones de diseño destacadas

- **Persistencia inmediata:** cada vez que el usuario agrega o modifica un país, el sistema guarda automáticamente el CSV, evitando pérdida de datos.
- **Validación de entradas:** la función `pedir_entero()` centraliza la validación numérica con soporte para valores opcionales, evitando duplicar código.

- **Búsqueda parcial:** se implementó coincidencia por subcadena (*termino in nombre.lower()*) para mayor comodidad del usuario.
- **Sin librerías externas:** el proyecto usa únicamente csv y os, módulos estándar de Python 3, cumpliendo con los requerimientos de la cátedra.

3. Dificultades y Conclusiones

3.1 Obstáculos encontrados

- La lectura del CSV requirió manejar casos borde: filas con campos vacíos, valores no numéricos en campos enteros y encoding de caracteres especiales (tildes, ñ). Se resolvió con bloques try/except dentro de la función *cargar_paises()*.
- La validación de entradas numéricas era repetitiva en múltiples funciones. Se resolvió centralizando esa lógica en la función utilitaria *pedir_entero()* con soporte para valores opcionales.
- El ordenamiento por nombre con caracteres especiales (acentos) funciona correctamente en Python 3 ya que las cadenas se comparan en Unicode por defecto.

3.2 Conclusiones

Este trabajo práctico permitió consolidar el uso de las estructuras de datos fundamentales de Python: listas y diccionarios. La combinación de ambas estructuras resultó natural para representar un dataset tabular, donde la lista actúa como contenedor de registros y cada diccionario encapsula los atributos de un país.

La modularización con funciones demostró ser clave para mantener el código legible y fácil de extender. Cada función tiene una responsabilidad clara, lo que facilitó la detección y corrección de errores durante el desarrollo.

El trabajo con archivos CSV evidenció la importancia de validar los datos de entrada, ya que un formato inesperado puede romper la carga del programa. El manejo de excepciones resultó fundamental para construir un sistema robusto.

4. Bibliografía / Webgrafía

- Python Software Foundation. (2024). *The Python Tutorial*. <https://docs.python.org/3/tutorial/>
- Python Software Foundation. (2024). *csv — CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (2024). *Data Structures — Lists and Dictionaries*. <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. (2024). *Built-in Functions: sorted(), max(), min(), sum()*. <https://docs.python.org/3/library/functions.html>
- Real Python. (2024). *Reading and Writing CSV Files in Python*. <https://realpython.com/python-csv/>
- Real Python. (2024). *Sorting Data With Python*. <https://realpython.com/python-sort/>

Repositorio del proyecto: <https://github.com/famoraes931-arch/tpi-gestion-paises>